

Standard library hardening

Document #: P3471R1
Date: 2024-11-21
Project: Programming Language C++
Audience: Library Evolution, SG23
Reply-to: Konstantin Varlamov
<varconst@apple.com>
Louis Dionne
<ldionne@apple.com>

§ 1 Introduction

This paper proposes introducing *standard library hardening* into the C++ Standard. Hardening allows turning some instances of undefined behavior in the standard library into a contract violation. This proposal is based on our experience implementing [hardening in libc++](#) and deploying it widely.

§ 2 Revision history

- R0: Initial proposal based on “terminating in an implementation-defined way”
- R1: Revision rebased on top of contract violations

§ 3 Motivation

There has been significantly increased attention to safety and security in C++ over the last few years, as exemplified by the well-known [White House report](#) and numerous recent security-related proposals.

While it is important to explore ways to make *new* code safer, we believe that the highest priority to deliver immediate real-world value should be to make *existing* code safer with minimal or no effort on behalf of users. Indeed, the amount of existing security-critical C++ code is so large that rewriting it or modifying it is both economically unviable and dangerous given the risk of introducing new issues.

There have been a few proposals accepted recently that eliminate some cases of undefined behavior in the core language. The standard library also contains many instances of undefined behavior, some of which is a direct source of security vulnerabilities; addressing those is often trivial, can be done with low overhead and almost no work on behalf of users.

In fact, at the moment all three major library implementations have some notion of a hardened or debug mode. This clearly shows interest, both from users and from implementers, in having a safer mode for the standard library. However, we believe these efforts would be vastly more useful if they were standardized and provided portable, cross-platform guarantees to users; as it stands, implementations differ in levels of coverage, performance guarantees and ways to enable the safer mode.

Finally, leaving security of the library to be a pure vendor extension fails to position ISO C++ as providing a credible solution for code bases with formal security requirements. We believe that formally requiring the basic safety guarantees that most implementations already provide in one way or another could make a significant difference from the point of view of anyone writing or following safety and security coding standards and guidelines.

§ 4 Deployment experience

All three major implementations provide vendor-specific ways of enabling library assertions as proposed in this paper, today.

- We have experience deploying hardening on Apple platforms in several existing codebases.
- Google recently published [an article](#) where they describe their experience with deploying this very technology to hundreds of millions of lines of code. They reported a performance impact as low as 0.3% and finding over 1000 bugs, including security critical ones.
- Google Andromeda published [an article](#) ~1 year ago about their successful experience enabling hardening.
- The libc++ maintainers have received numerous informal reports of hardening being turned on and helping find bugs in codebases.

Overall, standard library hardening has been a huge success, in fact we never expected so much success. The reception has been overwhelmingly positive and while the quality of implementation will never be perfect, we are working hard to expand the scope of hardening in libc++, to improve its performance and the user experience.

§ 5 Design overview

At a high level, this proposal consists of two parts:

- Annotate *some* existing library preconditions as *hardened*.
- Define the notion of a *hardened implementation*. In a hardened implementation, *hardened preconditions* are checked at runtime and violating them is a contract violation.

There are a few important aspects to the proposed design:

- In a *hardened implementation*, it is not possible to “turn off” these precondition checks by using the ignore semantic. Since we specify that violating a *hardened precondition* is a contract violation, we are already past the point where ignore could bypass the check.
- We only propose to formally check (some of the) existing function preconditions, not to add any new preconditions. As a drive-by, we also make a few existing preconditions more explicit in the specification (e.g. `vector::operator[]` only has implicit preconditions via iterator validity).
- Hardening deals exclusively with preconditions that must be checked at runtime. Thus, performance overhead is an important constraint.
- Hardening is intended to avoid requiring changes to an implementation’s ABI. In particular, the intent is to make it possible for implementations to allow mixing hardened and unhardened code in the same program, if they wish to do so.
- Hardening needs to be lightweight enough to be used in production environments. In our experience, a debug-only approach is not sufficient to really move the needle security-wise. Thus, the goal is to identify the minimal set of checks that would deliver the most value to users while requiring minimal overhead, with a particular focus on memory safety as a major source of security vulnerabilities. Checking *all* preconditions in the library is an explicit non-goal, and would be impossible for many semantic requirements anyway.

To reiterate the last point, an important design principle is that hardening needs to be lightweight enough for production use by a wide variety of real-world programs. In our experience in `libc++`, a small set of checks that is widely used delivers far more value than a more extensive set of checks that is only enabled by select few users. Thankfully, many of the most valuable checks, such as checking for out-of-bounds access in standard containers, also happen to be relatively cheap.

§ 6 Hardened preconditions

To specify hardening in the Standard, this proposal introduces the notion of a *hardened precondition*. A *hardened precondition* is a precondition that results in a contract violation in a *hardened implementation*. Adding hardening to the library largely consists of turning some of the existing preconditions into *hardened preconditions* in the specification. For example:

(24.7.2.2.6) Element access 23.7.2.2.6 [\[span.elem\]](#)

```
constexpr reference operator[](size_type idx) const;
```

¹ [Hardened](#) Preconditions: `idx < size()` is `true`.

In the initial proposal, we decide to focus on hardened preconditions that prevent out-of-bounds memory access, i.e., compromise the memory safety of the program. These are some of the most valuable for the user since they help prevent potential security vulnerabilities; many of them are also relatively cheap to implement. More hardened preconditions can potentially be added in the future, but the intent is for their number to be limited to keep hardening viable for production use. Specifically, the proposal is to add hardened preconditions to:

- Accessors of sequence containers, `std::span`, `std::mdspan`, `std::string`, `std::string_view` and other similar classes that might attempt to access non-existent elements (e.g. `back()` on an empty container or `operator[]` with an invalid index).
- Modifiers of sequence containers and string classes that assume the container to be non-empty (e.g. `pop_back()`).
- Accessors of `optional` and `expected` that expect the object to be non-empty.

In our experience, hardening all of these operations is trivial to implement and provides significant security value.

§ 7 Enabling hardening

Much like a freestanding implementation, the way to request a *hardened* implementation is left for the implementation to define. For example, similarly to `-ffreestanding`, we expect that most toolchains would provide a compiler flag like `-fhardened`, but other alternatives like a `-D_LIBCPP_HARDENING_MODE=<mode>` macro would also be conforming. If this proposal gets accepted, we expect implementations to converge on a portable mechanism. Other details like whether hardened implementations and non-hardened implementations can be mixed in different translation units are intentionally left unspecified, to avoid overconstraining the implementations.

§ 8 Relationship to Contracts, Profiles, and Erroneous Behavior

§ 8.1 Contracts

Contracts is a new (in-progress) language feature that allows expressing preconditions and much more, with a lot of flexibility on what happens when an assertion is not satisfied. In the latest revision of this paper, we decided to base *hardened preconditions* on contract violations for several reasons:

- It provides a unifying framework for specifying what should be done when “something goes wrong” both at the language and the library level. A unified mechanism is something that our current hardening clients have been repeatedly requesting.

- It provides flexibility for handling contract violations in a way that best suits the user. In particular, our deployment experience has made it clear that different users have different requirements, and that e.g. the weaker observe semantics can be extremely useful to allow deploying this at scale.

It is useful to note that we don't require implementations to actually implement these checks as contract assertions. Implementations are free to implement precondition checking however they see fit (e.g. a macro), however they are required to employ the contract violation handling mechanism when a precondition is not satisfied.

Note that if Contracts were to not be pursued anymore, this feature could easily be reworded in terms of a guaranteed termination in an implementation-defined way, or using Erroneous Behavior. We are not strongly attached to the exact mechanism used to implement this, but we find that Contracts is a nearly perfect fit.

§ 8.2 Profiles

The various Profiles proposals introduce a framework to specify sets of safety guarantees (such as a type safety profile or an invalidation profile). If profiles become a part of the Standard in the future, hardening can most likely be formulated as an additional profile; this would formalize how hardening is turned on and off.

However, we feel strongly that a hardening mode as specified in this paper standardizes existing practice and delivers value today without waiting for a larger and still experimental language feature.

§ 8.3 Erroneous Behavior

While Erroneous Behavior is a way to clearly mark some code as incorrect in the specification, it does not clearly specify *what* should happen in case of EB. For example, a conforming behavior for `vector::operator[]` being called out-of-bounds under EB would be to return the last element of the vector instead. While that is well-defined behavior, we feel that it is not especially useful behavior and that is certainly not what our users are looking for. In contrast, the Contracts facilities provide a well-defined and flexible framework to handle this.

§ 9 Proposed wording (partial)

Add a new paragraph to 4.1 [\[intro.compliance\]](#) after paragraph 7 as indicated:

- 7 Two kinds of implementations are defined: a *hosted implementation* and a *freestanding implementation*. A freestanding implementation is one in which execution may take place without the benefit of an operating system. A hosted implementation supports all the facilities described in this document, while a freestanding implementation supports the entire C++ language described in

[lex] through [cpp] and the subset of the library facilities described in [compliance].

- 8 Additionally, an implementation can be a *hardened implementation*. A hardened implementation is one in which violating a *hardened precondition* is a contract violation.

Add a new element to 16.3.2.4 [structure.specifications] after element 3.3 as indicated:

- (3.3) *Preconditions*: the conditions that the function assumes to hold whenever it is called; violation of any preconditions results in undefined behavior.
- (3.4) *Hardened Preconditions*: the conditions that the function assumes to hold whenever it is called; violation of any hardened preconditions results in a contract violation in a hardened implementation, and undefined behavior otherwise.

Modify 23.7.2.2.6 [span.elem] around paragraph 1 as indicated:

```
constexpr reference operator[](size_type idx) const;
```

- 1 *Hardened Preconditions*: `idx < size()` is `true`.
- 2 *Returns*: `*(data() + idx)`.
- 3 *Throws*: Nothing.

Modify 27.3.3.6 [string.view.access] around paragraph 1 as indicated:

```
constexpr const_reference operator[](size_type pos) const;
```

- 1 *Hardened Preconditions*: `pos < size()`.
- 2 *Returns*: `data_[pos]`.
- 3 *Throws*: Nothing.

Modify 23.2.4 [sequence.reqmts] around paragraph 117 as indicated:

```
a[n]
```

- 117 *Result*: reference; const_reference for constant a
- 118 *Hardened Preconditions*: `n < a.size()`;
- 119 *Effects*: Equivalent to: `return *(a.begin() + n);`
- 120 *Remarks*: Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

Modify 22.8.6.6 [expected.object.obs] paragraph 1 as indicated:

```
constexpr const T* operator->() const noexcept;  
  
constexpr T* operator->() noexcept;
```

- 1 *Hardened* *Preconditions*: `has_value()` is `true`.
- 2 *Returns*: `addressof(val)`.

Modify 22.8.6.6 [expected.object.obs] paragraph 3 as indicated:

```
constexpr const T& operator*() const & noexcept;  
  
constexpr T& operator*() & noexcept;
```

- 3 *Hardened* *Preconditions*: `has_value()` is `true`.
- 4 *Returns*: `val`.

Modify 22.8.6.6 [expected.object.obs] paragraph 5 as indicated:

```
constexpr T&& operator*() && noexcept;  
  
constexpr const T&& operator*() const && noexcept;
```

- 5 *Hardened* *Preconditions*: `has_value()` is `true`.
- 6 *Returns*: `std::move(val)`.

Modify 22.5.3.7 [optional.observe] paragraph 1 as indicated:

```
constexpr const T* operator->() const noexcept;  
  
constexpr T* operator->() noexcept;
```

- 1 *Hardened* *Preconditions*: `*this` contains a value.
- 2 *Returns*: `*val`.
- 3 *Remarks*: These functions are `constexpr` functions.

Modify 22.5.3.7 [optional.observe] paragraph 4 as indicated:

```
constexpr const T& operator*() const & noexcept;  
  
constexpr T& operator*() & noexcept;
```

- 4 *Hardened* *Preconditions*: `*this` contains a value.
- 5 *Returns*: `*val`.

⁶ *Remarks:* These functions are `constexpr` functions.

Modify 22.5.3.7 `[optional.observe]` paragraph 7 as indicated:

```
constexpr T&& operator*() && noexcept;
```

```
constexpr const T&& operator*() const && noexcept;
```

⁷ *Hardened* *Preconditions:* `*this` contains a value.

⁸ *Effects:* Equivalent to: `return std::move(*val);`

§ 10 Suggested polls

- Are we interested in standardizing some portable notion of hardening for the Standard Library (as opposed to letting these efforts be vendor extensions)?
- Are we happy with hardening being phrased in terms of contract violations?
- Are we happy with advancing this paper independently from Profiles proposals (understanding they may be reunited in the future)?